Two-Phase Commit (2PC)

an atomic commit across several databases

Phase 4 videos on YouTube

In one line: in 2PC a coordinator asks every participant to prepare (durably promise to commit), and commits only if all vote yes; it's atomic, but participants hold locks while waiting and block if the coordinator fails after they voted, which is why it's used within databases and rarely between microservices.

1 One transfer, two databases

Move $100 from an account in one database to an account in another. The debit and the credit must both happen, or neither. Two separate databases, one atomic outcome. **Two-phase commit** (2PC) is the classic protocol for that.

2 The protocol

Phase 1, prepare (voting): the coordinator sends `PREPARE` to every participant. Each one does all the work, writes it durably to its log, holds its locks, and votes **YES** (it promises it can commit, even after a crash) or **NO**. **Phase 2, commit:** if every vote is YES, the coordinator writes its decision to its own log and sends `COMMIT`; if any vote is NO or missing, it sends `ABORT`. Participants apply the decision and release their locks.

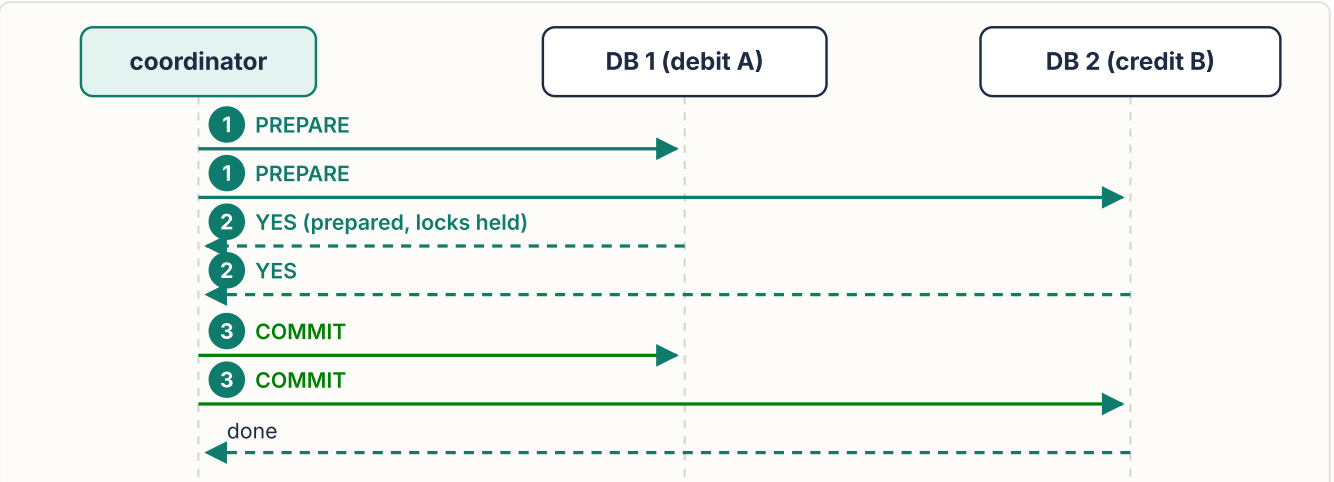


Figure 1. 2PC for a transfer. Once a participant votes YES, it can no longer decide on its own.

WORKED EXAMPLE: AN XA TRANSFER BETWEEN TWO POSTGRES DATABASES

The transaction manager debits account A in database 1 and credits account B in database 2, then runs `PREPARE TRANSACTION 'tx-81'` on both. Both succeed. It records 'commit tx-81' in its own log, then runs `COMMIT PREPARED 'tx-81'` on both. If database 2 had failed the credit (say, account closed), it would vote NO and both would run `ROLLBACK PREPARED`. If the manager crashes after the prepares, both transactions stay **in doubt**, holding row locks on A and B, until it recovers and reads its log.

3 Why 2PC has a bad reputation

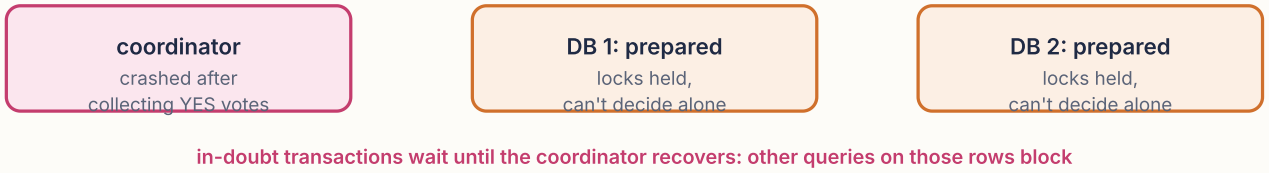


Figure 2. The blocking problem: prepared participants must wait for the coordinator's decision.

- **Blocking:** participants that voted YES can't commit or abort alone if the coordinator disappears.
- **Latency and locks:** at least two round trips, with locks held throughout, so throughput drops under contention.
- **Availability:** any one participant down means no transaction commits.
- **Coupling:** every participant must speak the same protocol; external APIs (a card processor, an email service) can't take part.

4 Where 2PC works well

Distributed databases such as Spanner and CockroachDB run 2PC across shards, but each participant and the coordinator are themselves **Raft or Paxos groups**, so they don't disappear, and the blocking problem mostly goes away. Kafka transactions use a similar two-phase idea to write to several partitions and commit consumer offsets atomically. 3PC (three-phase commit) tries to avoid blocking but isn't safe under network partitions, so it's rarely used.

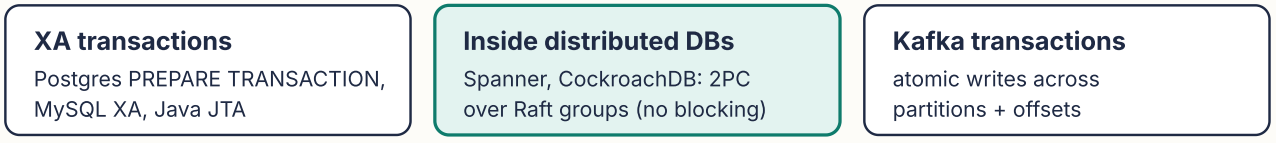


Figure 3. 2PC in practice: mostly inside systems that make the coordinator fault-tolerant.

COMMON MISTAKES

- Using XA between microservices owned by different teams.
- Forgetting to monitor and resolve in-doubt (prepared) transactions.
- Assuming 2PC protects calls to external, non-transactional APIs.

5 Key terms

Coordinator: drives the protocol and decides

Participant: a database taking part

Prepare / vote: durably promise to commit if told

In-doubt transaction: prepared, waiting for the decision

XA: standard interface for 2PC across resources

3PC: non-blocking variant, unsafe under partitions

6 Interview questions

> **Q1. Describe the two phases of 2PC.** BASIC
> Prepare: participants do the work durably, hold locks and vote yes or no. Commit: if all voted yes, the coordinator logs and sends commit; otherwise abort.

Q2. What is the main weakness of 2PC? BASIC

It blocks: if the coordinator fails after participants vote yes, they hold locks and can't decide until it recovers.

Q3. Would you use 2PC for order, payment and shipping microservices? SCENARIO

No: it couples the services, holds locks across network calls, and can't include external payment APIs. Use a saga with an outbox.

Q4. Your database has prepared transactions from last week holding locks. What happened and what do you do? SCENARIO

The coordinator crashed or lost track after prepare. Check its log for the decision and commit or roll back each one manually, then add monitoring for in-doubt transactions.

Q5. How do Spanner and CockroachDB avoid 2PC's blocking? TRICKY

The coordinator and participants are replicated with consensus, so the coordinator's decision survives node failures and another replica can finish the protocol.

Q6. Why doesn't 3PC solve the problem in practice? TRICKY

It assumes bounded network delays; under partitions, different groups can reach different decisions, breaking atomicity.

Next: SD-H-017 · Saga Pattern backendroadmap.com